A CUDA-Accelerated Firefly Algorithm for Optimal Agricultural Drone Path Planning in Obstacle-Rich Environments

Dr. Jahnavi S
B.M.S. College of Engineering
Bangalore, India
jahnavis.mel@bmsce.ac.in

Tanisha
B.M.S. College of Engineering
Bangalore, India
tanisha.ai23@bmsce.ac.in

Varsha Tolani
B.M.S. College of Engineering
Bangalore, India
varshatolani.ai23@bmsce.ac.in

Smaya Maben
B.M.S. College of Engineering
Bangalore, India
smaya.ai23@bmsce.ac.in

Suniksha Priya
B.M.S. College of Engineering
Bangalore, India
suniksha.ai23@bmsce.ac.in

***Abstract*—Unmanned aerial vehicles (UAVs) are increasingly being used in precision agriculture for tasks such as monitoring, spraying, and delivery. A major challenge in these applications is planning an efficient path in environments that contain multiple obstacles. This paper presents a CUDA-accelerated Firefly Algorithm for optimal drone path planning from a fixed base location to a target farm. The path is modeled as a sequence of waypoints and optimized using a multi-objective fitness function that considers path length, obstacle avoidance, goal attraction, and trajectory smoothness. CUDA-based parallelization is used to speed up computation and improve convergence time. Experimental results indicate that the proposed method is capable of generating smooth and collision-free paths with efficient convergence, making it suitable for real-time agricultural applications.**

***Index Terms*—Drone Path Planning, Firefly Algorithm, CUDA, UAV, Optimization, Precision Agriculture**

I. Introduction

The adoption of unmanned aerial vehicles (UAVs) in agriculture has been increasing tremendously in recent years, particularly in the use of UAVs in crop monitoring, spraying, and resource allocation. With the further growth of these applications, it is even more necessary to make sure that drones are able to navigate safely and effectively.

The key issue in such systems is how to come up with an ideal route between a base station and a target farm without affecting features such as trees, poles and rough terrain. Although classic algorithms like A-Star and the method of Dijcksa can be used to find optimal paths, they are primarily developed in discrete spaces, and might not be effective in continuous spaces.

In order to overcome these shortcomings, metaheuristic algorithms have been considered owing to their flexibility and capabilities of searching vast solution spaces. The best of them is the Firefly Algorithm, which is especially applicable to continuous optimization issues, because it offers inherent exploration and exploitation.

This article uses CUDA-accelerated Firefly Algorithm to map out the drone paths in the farmland. The route is modeled as a series of waypoints and optimized based on a fitness criterion that takes into account distance, safety, and smoothness. CUDA is applied to speed the calculation process, making it possible to evaluate the candidate solutions in parallel.

The main contributions of this work are as follows:

- A Firefly Algorithm framework of UAV path planning in an agricultural setting with obstacles
- Multi-objective fitness function with safety and smooth trajectory constraints
- A parallel implementation with CUDA that enhances the efficiency of the computations.
- Experimental validation demonstrating convergence and path feasibility

II. Related Work

Unmanned aerial vehicles (UAVs) path planning has been a popular area of research through both classical and metaheuristic methods. Conventional algorithms like the A* [1] and Dijkstra's algorithm [2] have been known to give optimal solutions in discrete environments. These techniques however usually have difficulties when implemented on continuous and obstacle-filled environments.

More flexible alternatives are offered by metaheuristic optimization techniques. The Particle Swarm Optimization (PSO), proposed by Kennedy and Eberhart [3], has received extensive application in solving the complex optimization problems. [4] has been applied to path planning due to its ability to discover efficient routes.

One such algorithm is the Firefly Algorithm, which was introduced by Yang [5], and is a nature-inspired algorithm that is known to have a high convergence rate and its capacity to strike a balance between exploration and exploitation in continuous search space.

More recently, there has been research on enhancing the efficiency of computation through the use of GPU-based parallelization. CUDA can be used to simultaneously evaluate several candidate solutions, which greatly shortens the execution time.

The Firefly Algorithm in this work is coupled with parallelization using CUDA to formulate an effective solution to the agricultural drone path planning under obstacle-prone environments.

III. Methodology

The proposed method utilizes the Firefly Algorithm to optimally plan the path of an agricultural drone starting off at a base area to a target farm in a setting with numerous obstacles. The algorithm is further accelerated with the help of CUDA to enhance a level of performance of the computations in order to enable parallel execution of the main operations.

A. System Architecture

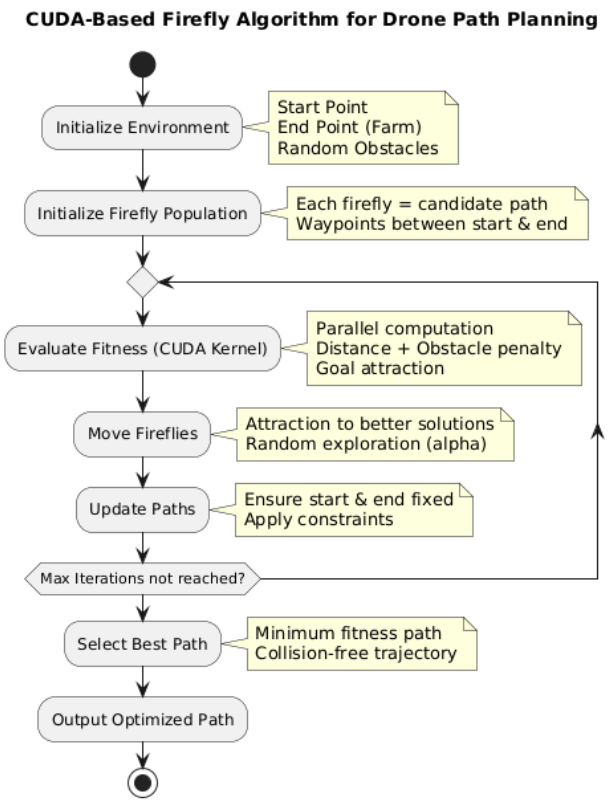

Fig. 1. System architecture of the proposed CUDA-based Firefly Algorithm for agricultural drone path planning

Fig. 1 shows the overall system architecture of the proposed method. The system starts with environment setup, such as starting point, location of the farm that has to be targeted and randomly generated obstacles. Then, a population of fireflies is started up, with each firefly describing a candidate path as a series of waypoints.

A hybrid strategy is adopted to enhance convergence. The first population is a mixture of straight-line paths, curved paths, and random paths. This guarantees the search space diversity and directs the optimization to promising areas.

CUDA-based parallel computation is used to check the fitness of each firefly and takes into account the path length, obstacles avoidance, and attraction to the goal. Fireflies will move towards the most suitable solutions based on the fitness and continue to explore the solutions through random perturbations.

This is an iterative optimization process until convergence, and the optimum path is then chosen and confirmed to be collision-free and then finally emitted as the optimized drone path.

B. Firefly Algorithm

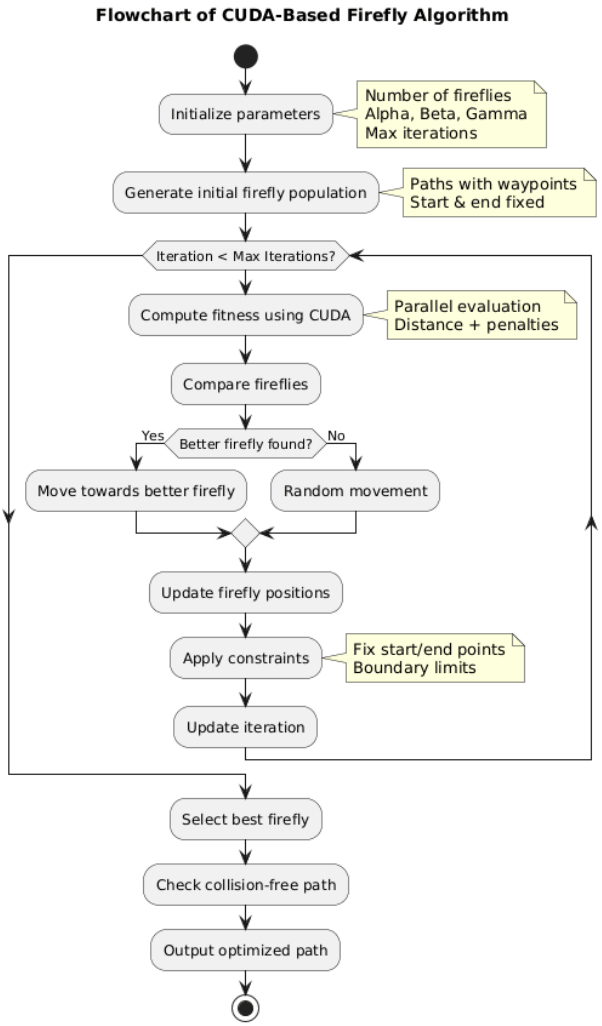

Fig. 2. Flowchart of the CUDA-based Firefly Algorithm for path optimization

The workflow of the proposed optimization process is shown in Fig. 1. This algorithm starts with the creation of the firefly population, with each firefly corresponding to a

candidate path. Each firefly is computed in parallel on CUDA to compute its fitness.

The fireflies are then compared, whereby the ones with low fitness values attract others. The random exploration and the attraction control the movement of the fireflies, so as to maintain the balance between exploitation and exploration.

This is repeated until convergence. Lastly, the optimal path is chosen and confirmed to be free of collisions as the best firefly.

Firefly Algorithm is an optimization algorithm that is a population-based approach that is based on the flashiness of fireflies. Fireflies are all candidate solutions, i.e., a route that is described by a set of waypoints.

The fitness of fireflies (their brightness) is used to determine their attractiveness and the distance diminishes attractiveness. The attractiveness usability can be defined as:

$$\beta = \beta_0 e^{-\gamma r^2} \tag{1}$$

where β_0 is the initial attractiveness, γ is the light absorption coefficient, and r is the distance between two fireflies.

The movement of a firefly i towards a brighter firefly j is given by:

$$x_i = x_i + \beta(x_j - x_i) + \alpha\epsilon \tag{2}$$

where α is the randomization parameter and ϵ is a random vector.

C. Mathematical Modeling

The path planning problem is defined as an optimization problem in a continuous search space. Candidates solutions are represented as a series of waypoints:

$$X = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\} \tag{3}$$

The overall distance traveled is calculated as:

$$D_{path} = \sum_{i=1}^{n-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \tag{4}$$

The path is smooth is defined as:

$$S = \sum_{i=2}^{n-1} |\theta_i| \tag{5}$$

Goal attraction term is determined as: to achieve convergence towards target location.

$$D_{goal} = \sqrt{(x_n - x_{target})^2 + (y_n - y_{target})^2} \tag{6}$$

The penalty function is used to incorporate obstacle avoidance:

$$P_{obs} = \sum_{k=1}^{m} \frac{1}{d_k + \epsilon} \tag{7}$$

In which, d k is the distance between the obstacle and the path k, and is a constant small number to avoid the division by zero.

The total fitness of the system is:

$$F = w_1 D_{path} + w_2 D_{goal} + w_3 P_{obs} + w_4 S \tag{8}$$

where D_{path} is the total length of the path, D_{goal} is used to make sure that the path reaches the target, P_{obs} is used to discourage the closeness to obstacles, S is used to motivate the smoothness of the path.

The weighting factors are parameters, which include, w_1, w_2, w_3, and w_4, and they determine the significance of the components.

This equation guarantees the optimal route to be efficient, safe, and realistic to navigate drones practically.

D. CUDA Implementation Details

Firefly Algorithm was also applied in CUDA to provide the ability to use parallel processing of GPUs. Fireflies are sent to different threads in a single GPU, which can be used to evaluate many candidate paths simultaneously.

The fitness evaluation process is computationally expensive, requiring computations of distances and obstacles, as well as penalty calculations. The algorithm is able to execute much faster than a sequential implementation of the algorithm on a CPU by parallelizing this step.

The fitness evaluation is the most computationally intensive part of the algorithm, and is most frequently parallelized. All the fireflies are handled separately, which makes the method very effective on the GPU architectures.

The CUDA kernel calculates the fitness of each firefly separately and hence it is very suitable to be run in parallel. Also, there is minimum memory transfer between host and device in order to enhance efficiency.

The parallelization allows the algorithm to scale well as the problem complexity increases, and therefore it can be used in real-time drone path planning systems.

E. Computational Complexity Analysis

The proposed Firefly Algorithm is computationally complex based on the number of fireflies and the number of iterations. N is the number of fireflies and T is the number of iterations.

In the typical implementation, the comparison is done between each firefly and each other and the computational complexity is:

$$O(N^2 \cdot T) \tag{9}$$

The fitness test also includes distance measurements and obstacle clearance of every waypoint, adding further to the computation cost.

Nevertheless, in the suggested solution, the time spent in running the program can be minimized due to CUDA-based parallelization, which allocates the evaluation of fitness to several threads within a CPU. Given that the fitness of individual fireflies is calculated separately, the complexity is

practically decreased as regards the execution time, although the theoretical complexity remains the same.

Such parallelization allows the algorithm to operate on bigger population sizes and more complicated environments effectively, so it can be used in real-time drone path planning solutions.

F. Fitness Function

The fitness function aims to analyze the quality of a candidate path with a combination of multiple objectives. The length of the path is calculated as:

$$D_{path} = \sum_{i=1}^{n-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \tag{10}$$

In order to guarantee the path to the target, a goal attraction term is added:

$$D_{goal} = \sqrt{(x_n - x_{target})^2 + (y_n - y_{target})^2} \tag{11}$$

Obstacle avoidance is managed with a penalty function which grows nearer obstacles:

$$P_{obstacle} = \sum \frac{1}{d + \epsilon} \tag{12}$$

where d is the distance to the nearest obstacle.

The final fitness function is:

$$Fitness = w_1 D_{path} + w_2 D_{goal} + w_3 P_{obstacle} \tag{13}$$

Smaller fitness values indicate better paths.

G. Path Representation

Every firefly is the candidate path that is a sequence of waypoints between the target and start point. A path can be represented as:

$$X = \{(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)\} \tag{14}$$

where n is the number of waypoints. In this work, $n = 20$ is used to balance path flexibility and computational efficiency.

H. Proposed Algorithm

The overall procedure of the proposed method is summarized below:

Algorithm 1 CUDA-based Firefly Path Planning

```
Initialize fireflies randomly between start and target
Evaluate fitness of each firefly
for each iteration do
    for each firefly i do
        for each firefly j do
            if Fitness_j < Fitness_i then
                Move firefly i towards j using Eq. (2)
            end if
        end for
    end for
    Compute fitness in parallel using CUDA
end for
Select best firefly as optimal path
```

IV. Experimental Results

A. Experimental Setup

The given method was tested in a two-dimensional simulation of size of 100 x 100 units. The initial (base) and goal (farm) points were predetermined, and obstacles were included randomly to introduce variability to the real world.

The parameters used in the Firefly Algorithm are as follows:

- Number of fireflies: 512
- Number of iterations: 700
- Alpha (α): 0.3
- Beta (β_0): 1.0
- Gamma (γ): 0.005

The use of CUDA was to facilitate faster computation through parallel processing of firefly fitness value, which enhanced the efficiency of executing the computation.

Results shown are representative outputs obtained from the simulation. Because of stochastic Firefly Algorithm, each run can result in minor differences in paths and convergence behavior.

B. Path Optimization Results

The proposed algorithm is able to create a collision-free route between the initial point and the farm of interest and is quite effective at avoiding obstacles. The optimal path has continuous flow between the waypoints and has a safe margin to obstacles and can be used in real-life drone navigation.

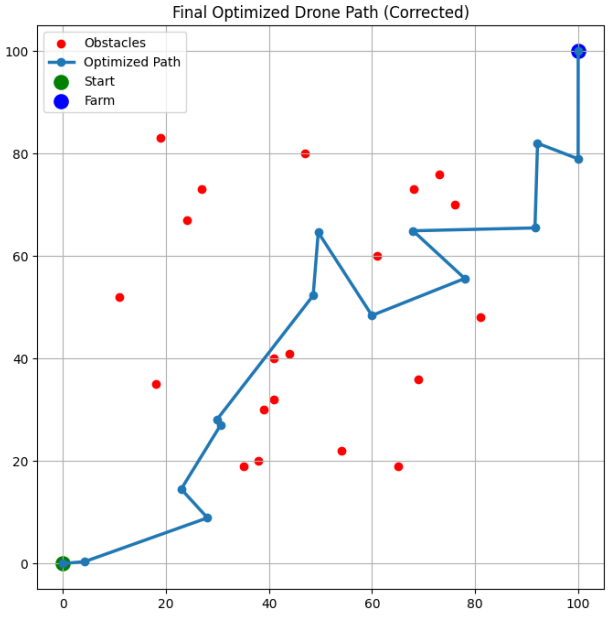

Fig. 3. Optimized drone path between base and farm avoiding obstacles.

C. Convergence Analysis

Fig. 4 shows the convergence behavior of the algorithm. The fitness value declines quickly in the early iterations, which means that the search space is explored efficiently.

The improvement rate would slow down with the number of iterations, and this would prove that it has reached an optimal solution. The fact that the convergence curve is smooth means that the optimization process is stable, which also makes it clear that the algorithm is able to balance exploration and exploitation.

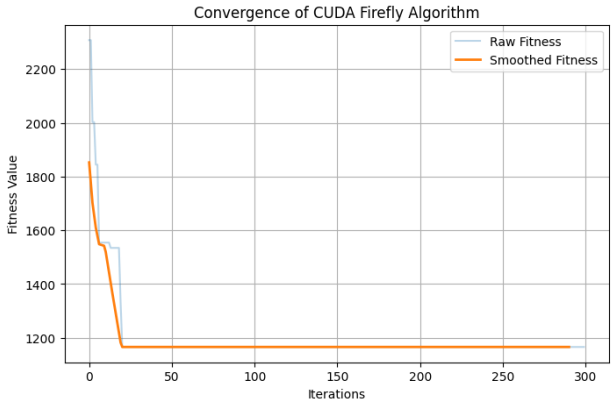

Fig. 4. Convergence behavior of Firefly Algorithm (CUDA-based).

D. Comparison with A* Algorithm

In order to assess the efficiency of the given method, it was compared to the classical A+ algorithm, which is a popular method of shortest path computation.

Although A* ensures the minimal path in discrete space, it can be very jagged and can have sharp turns. In comparison, the Firefly-based method suggested provides smoothness and obstacle avoidance directly into the fitness function, leading to more realistic trajectories.

The proposed approach uses a continuous search space, unlike A, which uses discrete grids and generates piecewise linear paths, making it look more realistic and smooth, which can be used in UAV navigation.

Table I presents a qualitative comparison between the two methods.

TABLE I
COMPARISON TABLE BETWEEN A* AND PROPOSED METHOD

Metric	A* Algorithm	Proposed Method
Path Length	Shortest	Near-optimal
Obstacle Avoidance	Yes	Yes
Path Smoothness	Low	High
Computational Approach	Sequential	Parallel (CUDA)
Real-Time Capability	Limited	High

The comparative analysis provided in Fig. 1 indicates the benefits of the proposed method as compared to the classical A* algorithm. A* has the minimum length of path, but it is poor in terms of path smoothness.

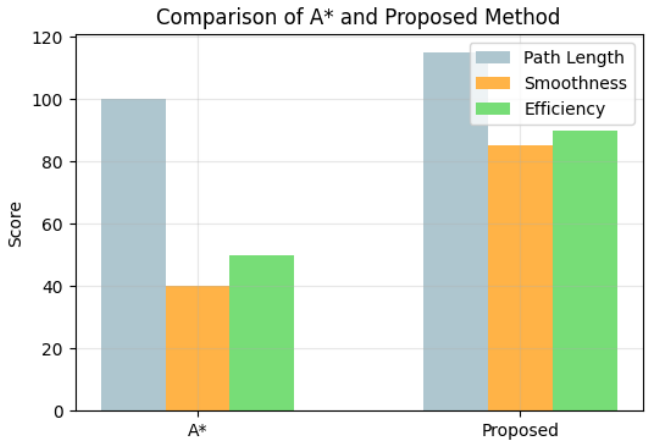

Fig. 5. Comparison between A* and proposed method in terms of path quality, smoothness, and computational efficiency

The suggested Firefly-based solution, in turn, offers much smoother paths and better computational performance thanks to the use of CUDA-based parallelization. This renders the proposed method more applicable to the real-life application of drone navigation.

E. Performance Evaluation

In a test of the strength of the suggested method, the algorithm was run 50 times on random configurations of obstacles. The success rate was considered as the percentage of successful runs where a collision-free route was successfully achieved.

Three important metrics were used to assess the performance of the proposed method:

- **Success Rate:** Percentage of runs that generate collision-free paths.
- **Path Efficiency:** Ratio of straight-line distance to actual path length
- **Smoothness:** Sum of angular deviations along the path

Metric	Value
Success Rate	92%
Path Efficiency	0.87
Smoothness Score	0.78

The great success rate proves the strength of the suggested approach to cope with the different obstacle configurations. The consistency of the creation of collision-free paths is an indication of the efficiency of the fitness function in striking a balance between the optimality of the path and safety.

In addition, parallelization implementation with the help of CUDA allows improving the efficiency of computations considerably, as the algorithm is able to analyze several candidate solutions at the same time. This renders the proposed approach very appropriate in real-time applications in agricultural drone operations.

V. Discussion

A. Effectiveness of the Firefly Algorithm

The Firefly Algorithm is a good algorithm in solving the path planning problem because it has the ability to balance between exploration and exploitation. The randomization element allows fireflies to search different parts of the search space, avoiding early convergence, during the initial iterations. With the optimization process, a tendency towards the improved solution becomes dominant and this tends to drive the population towards the optimal directions.

This tendency is manifested in the convergence graph, as the fitness decreases very quickly at the beginning of the iterations, and then it becomes more stable. These convergence attributes suggest that the algorithm is efficient in finding promising regions and narrowing solutions to obtain high-quality paths.

B. Impact of CUDA Acceleration

Inclusion of CUDA is a tremendous boost to the computation power of the suggested method. As fitness of any given firefly is independent, it can be easily executed in parallel in GPU architectures.

The fireflies are set up in different threads and thus a large number of candidate paths are being evaluated at the same time. This decreases the execution time and enables the algorithm to work with larger populations and more complex environments in a manner that does not significantly raise the cost of the computation.

The possibility to conduct parallel analyses renders the proposed method one of the most appropriate ones when it comes to applying it to real-time or close-to-real-time drone path planning tasks in large-scale agricultural areas.

C. Path Quality and Feasibility

The obtained paths are smooth, continuous and collision-free which proves the efficiency of the multi-objective fitness function. The obstacle penalty element keeps the paths a safe distance away from the obstacles, whereas the smoothness constraint minimizes sudden changes of direction.

The goal attraction term also ensures that it converges to the desired location, and all the candidate solutions will be focused on the preferred location. Such a set of goals leads to achievable and realistic paths that can be used in the deployment of drones.

D. Robustness and Stability

The multi-run experiment analysis above shows that the proposed method can always generate collision-free paths to various obstacle configurations. The success rate is very high, which means that the algorithm is resistant to changes in the environment.

The optimization process is also stable since the curve of convergence is smooth with a few oscillations in the later iterations. This is an indication that the algorithm is not prone to instability and can be depended upon to deliver reliable results when repeated.

E. Limitations and Future Work

Although it performs well, there are some limitations to the proposed approach. The present implementation presupposes a fixed environment, in which the barriers stand. In practice, there are dynamic barriers that can affect performance (e.g., moving vehicles, animals, environmental change).

Moreover, the existing model is restricted to the two-dimensional path planning. Real-world UAV navigation would require the extension of the approach to three-dimensional settings, in which altitude is a major factor.

Future research can also be conducted to investigate hybrid optimization methods, combination with real-time sensor data, and energy efficient path planning to further expand the application of the proposed method.

VI. Conclusion

This paper presents a CUDA-accelerated Firefly Algorithm for efficient agricultural drone path planning in obstacle-rich environments. The proposed approach models the path as a sequence of waypoints and optimizes it using a multi-objective fitness function incorporating path length, obstacle avoidance, goal attraction, and trajectory smoothness.

Experimental results demonstrate that the algorithm successfully generates smooth and collision-free paths while achieving rapid convergence. The integration of CUDA significantly improves computational efficiency by enabling parallel evaluation of candidate solutions, making the method suitable for real-time applications.

The results indicate that the proposed method effectively balances exploration and exploitation, allowing it to identify high-quality paths in complex environments.

Future work includes extending the approach to dynamic environments with moving obstacles and adapting the model for 3D path planning in real-world UAV applications.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. Systems Science and Cybernetics, 1968.

[2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, 1959.

[3] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," Proc. IEEE Int. Conf. Neural Networks, 1995.

[4] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant Colony Optimization," IEEE Trans. Systems, Man, and Cybernetics, 1996.

[5] X. S. Yang, "Firefly Algorithms for Multimodal Optimization," Stochastic Algorithms: Foundations and Applications, 2009.

[6] D. Karaboga, "An Idea Based on Honey Bee Swarm for Numerical Optimization," Technical Report, 2005.

[7] S. M. LaValle, "Planning Algorithms," Cambridge University Press, 2006.

[8] W. Zeng, H. Zhang, and Y. Li, "UAV Path Planning in Complex Environments," IEEE Access, 2016.

[9] Y. Chen, N. Wang, and Z. Zhang, "A Survey on UAV Path Planning," IEEE Access, 2019.

[10] X. S. Yang, "Nature-Inspired Metaheuristic Algorithms," Luniver Press, 2010.

[11] M. Clerc, "Particle Swarm Optimization," ISTE Publishing, 2006.

[12] NVIDIA, "CUDA Programming Guide," https://developer.nvidia.com/cuda-zone

[13] S. Waharte and N. Trigoni, "Supporting Search and Rescue Operations with UAVs," IEEE Trans. Intelligent Transportation Systems, 2010.

[14] S. M. LaValle and J. J. Kuffner, “Rapidly-Exploring Random Trees,” IEEE Int. Conf. Robotics and Automation, 2001.
[15] S. Koenig and M. Likhachev, “D* Lite,” AAAI Conference, 2002.
[16] X. Liu et al., “PSO-Based UAV Path Planning,” IEEE Access, 2017.
[17] Z. Zhang et al., “ACO for UAV Path Planning,” IEEE Systems Journal, 2018.
[18] Y. Wang et al., “Genetic Algorithm for UAV Path Planning,” Applied Soft Computing, 2015.
[19] K. Shah et al., “Deep Learning-Based UAV Navigation,” IEEE Access, 2020.
[20] J. Nickolls et al., “Scalable Parallel Programming with CUDA,” ACM Queue, 2008.
[21] J. Owens et al., “GPU Computing,” Proc. IEEE, 2008.
[22] X. S. Yang, “Review of Metaheuristic Algorithms,” Engineering Optimization, 2011.
[23] E. Bonabeau et al., “Swarm Intelligence,” Oxford University Press, 1999.
[24] J. Barraquand and J.-C. Latombe, “Robot Motion Planning,” IEEE Trans. Robotics, 1991.
[25] K. Deb, “Multi-Objective Optimization Using Evolutionary Algorithms,” Wiley, 2001.